

DQN với experience replay

DEEP REINFORCEMENT LEARNING BẰNG PYTHON



Timothée Carayol

Principal Machine Learning Engineer,
Komment

Giới thiệu về experience replay

- DQN tối giản chỉ học từ trải nghiệm mới nhất
 - Các cập nhật liên tiếp tương quan mạnh
 - Tác tử dễ quên
- Giải pháp: Experience Replay
 - Lưu trải nghiệm vào buffer
 - Mỗi bước, học từ một batch ngẫu nhiên các trải nghiệm cũ



Hàng đợi hai đầu (Deque)

```
from collections import deque
```

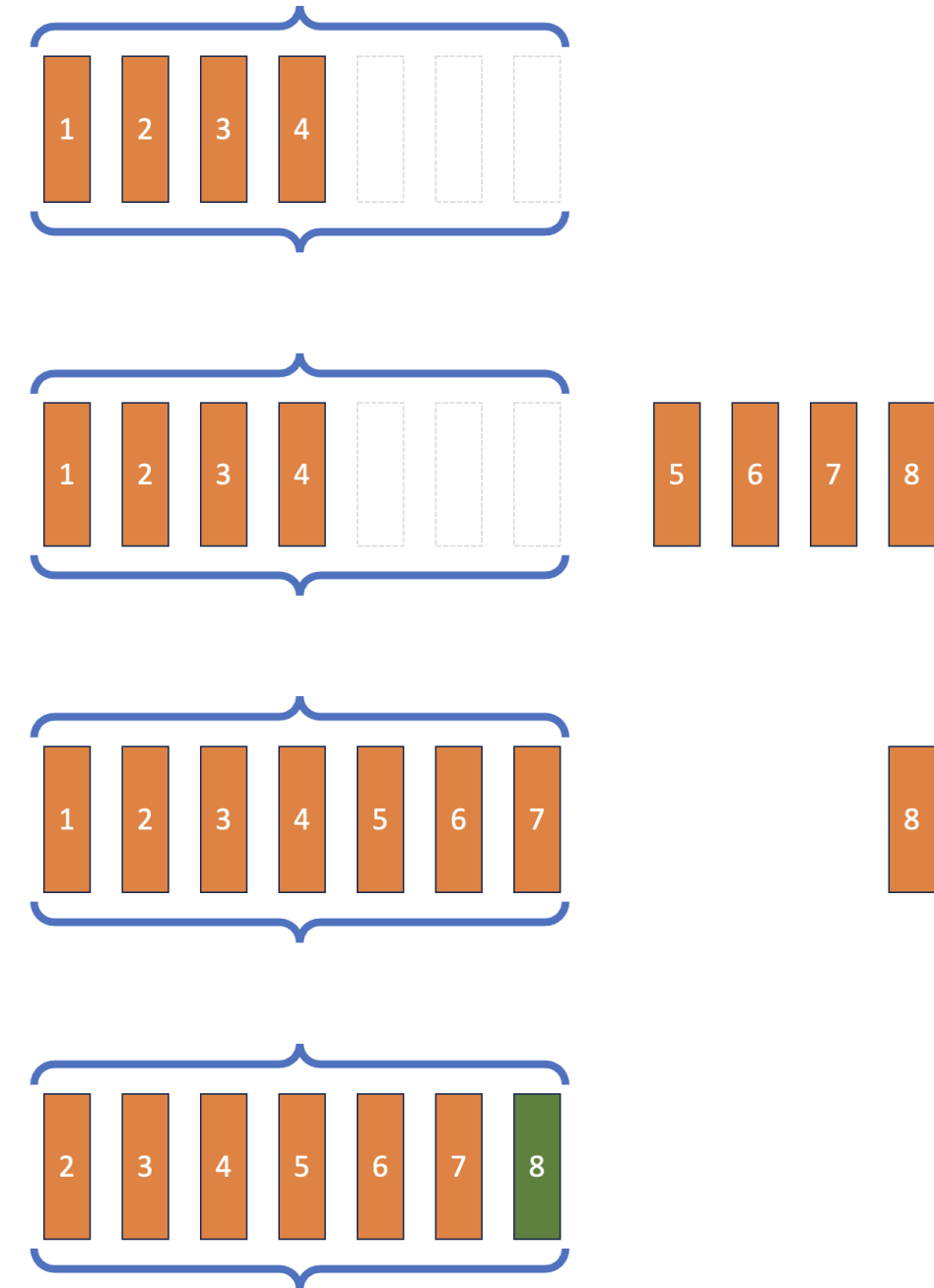
```
# Khởi tạo với dung lượng giới hạn
```

```
buffer = deque([1, 2, 3, 4], maxlen=7)
```

```
# Thêm vào phía phải
```

```
buffer.extend([5, 6, 7, 8])
```

- Vượt dung lượng, phần tử cũ nhất sẽ bị loại



Triển khai Replay Buffer

```
import random

class ReplayBuffer:
    def __init__(self, capacity):
        self.memory = deque([], maxlen=capacity)

    def push(self, state, action,
            reward, next_state, done):
        experience_tuple = (state, action,
                           reward, next_state, done)
        self.memory.append(experience_tuple)

    def __len__(self):
        return len(self.memory)

...
```

- Replay memory: `deque` dung lượng giới hạn
- `.push()` :
 - Trải nghiệm là bộ chuyển tiếp
 - Thêm trải nghiệm vào buffer
 - Đầy dung lượng: loại bỏ trải nghiệm cũ nhất

Triển khai Replay Buffer

```
...
def sample(self, batch_size):
    batch = random.sample(self.memory, batch_size)
    states, actions, rewards, next_states, dones = (
        zip(*batch))

    states_tensor = torch.tensor(
        states, dtype=torch.float32)
    ... # repeat identically for
        # rewards, next_states, dones
    actions_tensor = torch.tensor(
        actions, dtype=torch.long).unsqueeze(1)

    return states_tensor, actions_tensor,
    rewards_tensor, next_states_tensor, dones_tensor
```

- Lấy ngẫu nhiên từ trải nghiệm quá khứ
- `batch` : từ danh sách các bộ chuyển tiếp...
- ...thành bộ các danh sách...
- ...thành bộ các tensor PyTorch

Tích hợp Experience Replay trong DQN

1. Trước vòng lặp huấn luyện:

```
replay_buffer = ReplayBuffer(10000)
```

2. Trong vòng lặp, sau khi chọn hành động:

```
replay_buffer.push((state, action,
                    reward, next_state, done))

if len(replay_buffer) >= batch_size:
    states, actions, rewards, next_states, dones = (
        replay_buffer.sample(batch_size))
    q_values = (
        q_network(states).gather(1, actions).squeeze(1))
    next_states_q_values = q_network(next_states).amax(1)
    target_q_values = (
        rewards +
        gamma * next_states_q_values * (1-dones))
    loss = nn.MSELoss()(target_q_values, q_values)
```

- Khởi tạo replay buffer
- Đẩy chuyển tiếp mới nhất vào buffer

Nếu độ dài buffer $\geq$ batch_size :

- Lấy một batch ngẫu nhiên và tính loss
- Cách tính loss về cơ bản không đổi
- Mean Squared Bellman Error trên batch của replay memory
 - Học ổn định và hiệu quả hơn

Ayo berlatih!

DEEP REINFORCEMENT LEARNING BẰNG PYTHON

Thuật toán DQN hoàn chỉnh

DEEP REINFORCEMENT LEARNING BẰNG PYTHON

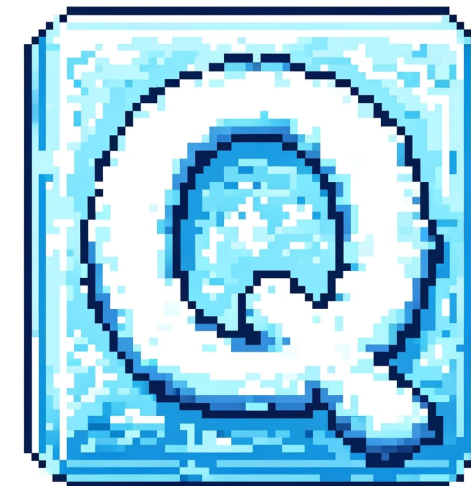


Timothée Carayol

Principal Machine Learning Engineer,
Komment

Thuật toán DQN

- Đã học DQN với Replay Kinh nghiệm
- Gần với DQN bản đầu tiên (2015)
- Còn thiếu hai thành phần:
 - Epsilon-greediness -> tăng khám phá
 - Fixed Q-targets -> học ổn định hơn

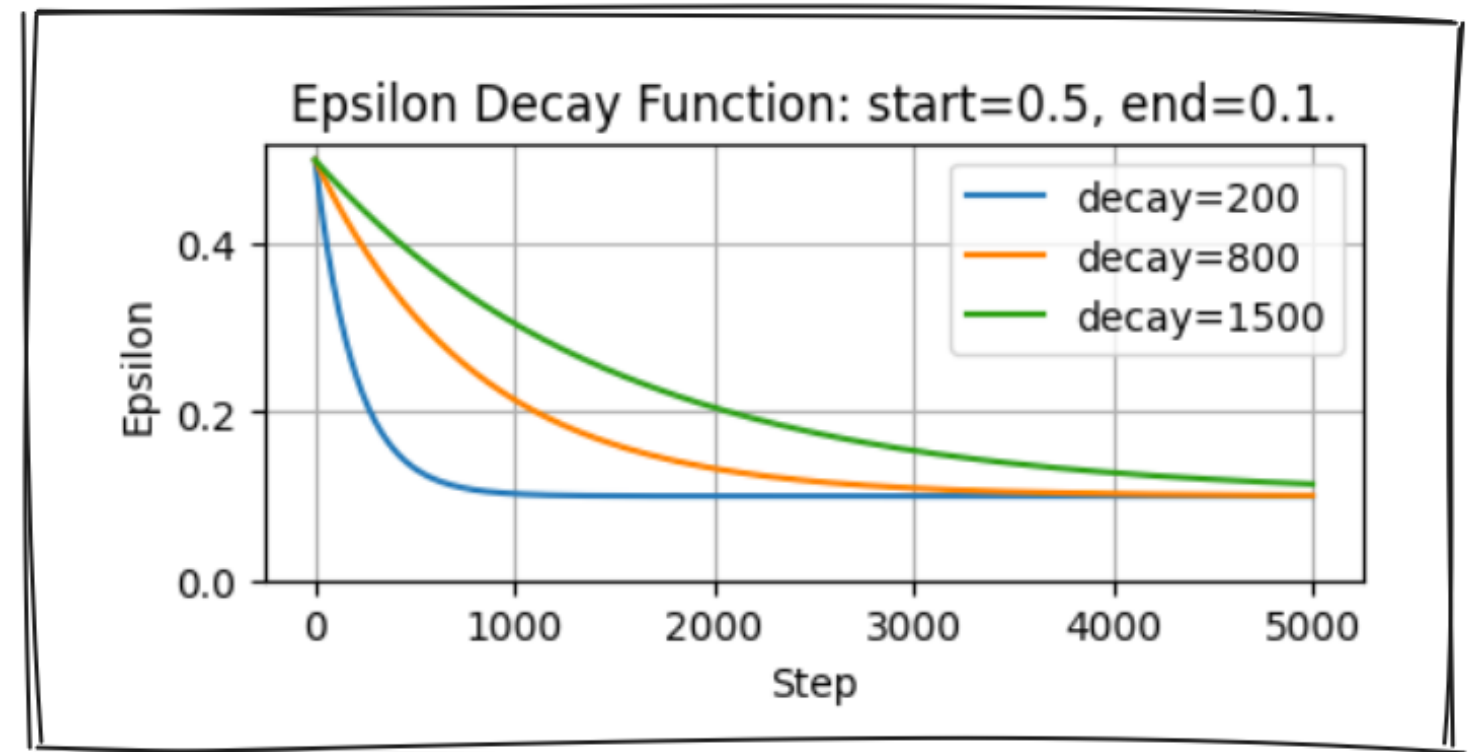


Epsilon-greediness trong DQN

- Triển khai epsilon-greediness suy giảm trong `select_action()`

```
def select_action(q_values, step, start, end, decay):  
    # Tính ngưỡng cho bước này  
    epsilon = (  
        end + (start-end) * math.exp(-step / decay))  
    # Lấy số ngẫu nhiên trong [0, 1]  
    sample = random.random()  
    if sample < epsilon:  
        # Trả về chỉ số hành động ngẫu nhiên  
        return random.choice(range(len(q_values)))  
    # Trả về chỉ số hành động có Q-value lớn nhất  
    return torch.argmax(q_values).item()
```

- $\epsilon = end + (start - end) \cdot e^{-\frac{step}{decay}}$
- Chọn hành động ngẫu nhiên với xác suất ϵ
- Chọn hành động giá trị cao nhất với xác suất $1 - \epsilon$



Fixed Q-targets

- Trong Bellman Error:
 - Q-Network dùng cho cả Q-Value và TD-Target
 - Mất ổn định do mục tiêu thay đổi
- Thêm target network để ổn định mục tiêu

Bellman Error

$$\left(r_{t+1} + \gamma \max_{a_{t+1}} \hat{Q}(s_{t+1}, a_{t+1}) \right) - \hat{Q}(s_t, a_t)$$

TD target

Current Q value estimate

Bellman Error (fixed Q-targets)

$$\left(r_{t+1} + \gamma \max_{a_{t+1}} \hat{Q}_{target}(s_{t+1}, a_{t+1}) \right) - \hat{Q}_{online}(s_t, a_t)$$

TD target

Current Q value estimate

Triển khai fixed Q-targets

```
online_network = QNetwork(state_size, action_size)
target_network = QNetwork(state_size, action_size)
target_network.load_state_dict(
    online_network.state_dict())

def update_target_network(
    target_network, online_network, tau):
    target_net_state_dict = target_network.state_dict()
    online_net_state_dict = online_network.state_dict()
    for key in online_net_state_dict:
        target_net_state_dict[key] = (
            online_net_state_dict[key] * tau +
            target_net_state_dict[key] * (1 - tau))
    target_network.load_state_dict(
        target_net_state_dict)

    return None
```

- Ban đầu Online Network = Target Network
- State dict của mạng chứa toàn bộ trọng

```
fc1.weight :
    tensor(
      [[-0.5059, -0.6368],
       [-0.4193,  0.6154]]
    )
fc1.bias :
    tensor([ 0.2465, -0.5916])
fc2.weight :
    tensor(
      [[-0.1208, -0.2308],
       [-0.2805,  0.5415]]
    )
```

số:

- Mỗi bước, mọi trọng số của Target Network tiến gần Online Network một chút

Tính loss với fixed Q-targets

```
# Trong vòng lặp trong, sau khi chọn hành động
if len(replay_buffer) >= batch_size:
    states, actions, rewards, next_states, dones =
        replay_buffer.sample(64)
    q_values = (online_network(states)
                .gather(1, actions).squeeze(1))

    with torch.no_grad():
        next_q_values = (
            target_network(next_states).amax(1))
        target_q_values = (
            rewards + gamma * next_q_values * (1 - dones))
    loss = torch.nn.MSELoss()(target_q_values, q_values)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    update_target_network(
        target_network, online_network, tau)
```

- Q-values dùng `online_network`
- Target Q-values dùng `target_network`
- Dùng `torch.no_grad()` để tắt gradient cho target Q-values
- Vẫn dùng Mean Squared Bellman Error để tính loss
- Dùng `update_target_network()` để cập nhật chậm `target_network`

Ayo berlatih!

DEEP REINFORCEMENT LEARNING BẰNG PYTHON

Double DQN

DEEP REINFORCEMENT LEARNING BẰNG PYTHON

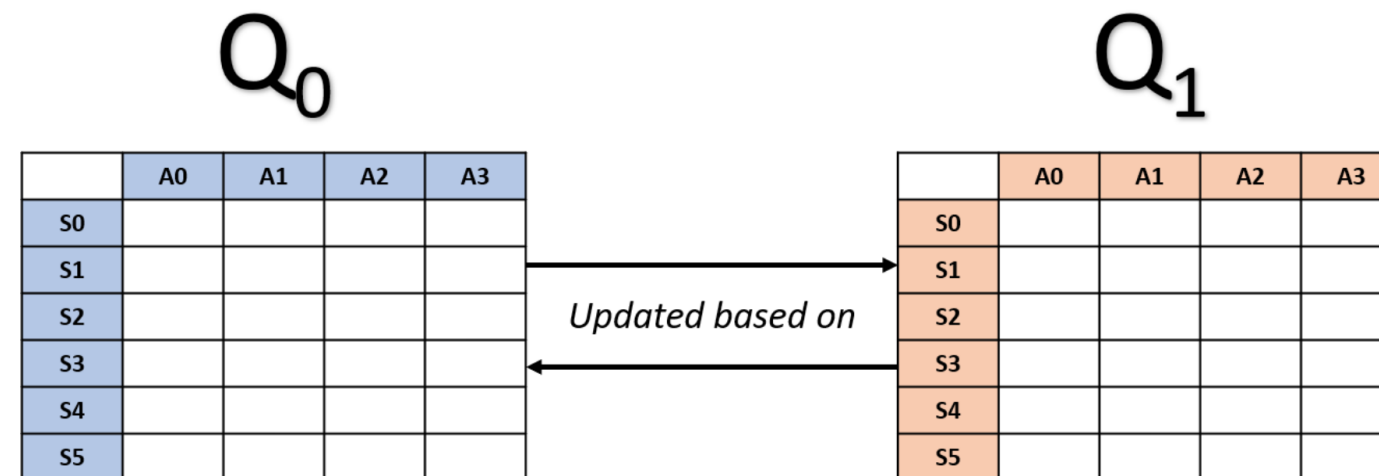


Timothée Carayol

Principal Machine Learning Engineer,
Komment

Double Q-learning

- Q-learning đánh giá cao quá mức Q-value, làm giảm hiệu quả học
- Do thiên lệch tối đa hóa
- Double Q-learning khử thiên lệch bằng cách tách chọn hành động và ước lượng giá trị



$$\max_a = \operatorname{argmax}(Q_1(s'))$$

$$\underbrace{Q_1(s, a)}_{\text{New Q value}} = (1 - \alpha) \underbrace{Q_1(s, a)}_{\text{Old Q value}} + \alpha [r + \gamma \underbrace{Q_0(s', \max_a)}_{\text{Maximum Q value given the next state}}]$$

Ý tưởng đằng sau DDQN

- Bắt đầu từ DQN hoàn chỉnh (với Q-target cố định)
- Trong DQN, TD target:
 - Chọn hành động: mạng target
 - Ước lượng giá trị: mạng target
- Trong DDQN, TD target:
 - Chọn hành động: mạng online
 - Ước lượng giá trị: mạng target
- Không hoàn toàn là double Q-learning (không thay phiên Q-network)
- Đạt phần lớn lợi ích, thay đổi tối thiểu

TD Error (DQN with fixed Q-targets)

$$\left(r_{t+1} + \gamma \max_{a_{t+1}} \hat{Q}_{target}(s_{t+1}, a_{t+1}) \right) - \hat{Q}_{online}(s_t, a_t)$$

TD target

Current Q value estimate

TD Error (DDQN)

$$\left(r_{t+1} + \gamma \max_{\tilde{a}} \hat{Q}_{target}(s_{t+1}, \tilde{a}) \right) - \hat{Q}_{online}(s_t, a_t)$$

TD target

Current Q value estimate

$$\text{with } \tilde{a} = \arg \max_{a_{t+1}} \hat{Q}_{online}(s_{t+1}, a_{t+1})$$

Cài đặt Double DQN

DQN:

```
... # instantiate online and target networks
q_values = (online_network(states)
            .gather(1, actions).squeeze(1))
with torch.no_grad():
    #
    #
    next_q_values = (target_network(next_states)
                    .amax(1))
    target_q_values = (rewards +
                      gamma * next_q_values * (1 - dones))
loss = torch.nn.MSELoss()(q_values, target_q_values)
... # gradient descent
... # target network update
```

DDQN:

```
... # instantiate online and target networks
q_values = (online_network(states)
            .gather(1, actions).squeeze(1))
with torch.no_grad():

    target_q_values = (rewards +
                      gamma * next_q_values * (1 - dones))
loss = torch.nn.MSELoss()(q_values, target_q_values)
... # gradient descent
... # target network update
```

Cài đặt Double DQN

DQN:

```
... # instantiate online and target networks
q_values = (online_network(states)
            .gather(1, actions).squeeze(1))
with torch.no_grad():
    next_actions = (target_network(next_states)
                    .argmax(1).unsqueeze(1))
    next_q_values = (target_network(next_states)
                    .gather(1, next_actions).squeeze(1))
    target_q_values = (rewards +
                      gamma * next_q_values * (1 - dones))
loss = torch.nn.MSELoss()(q_values, target_q_values)
... # gradient descent
... # target network update
```

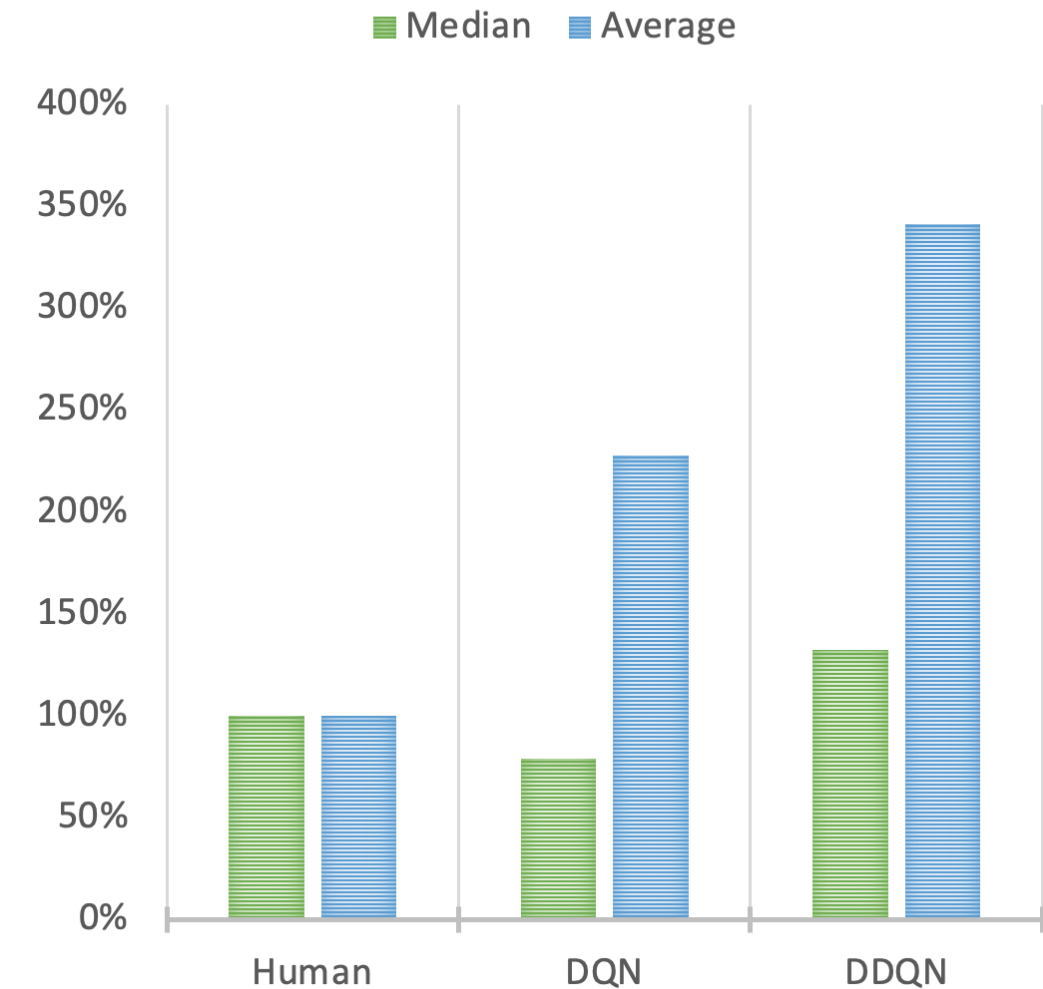
DDQN:

```
... # instantiate online and target networks
q_values = (online_network(states)
            .gather(1, actions).squeeze(1))
with torch.no_grad():
    next_actions = (online_network(next_states)
                    .argmax(1).unsqueeze(1))
    next_q_values = (target_network(next_states)
                    .gather(1, next_actions).squeeze(1))
    target_q_values = (rewards +
                      gamma * next_q_values * (1 - dones))
loss = torch.nn.MSELoss()(q_values, target_q_values)
... # gradient descent
... # target network update
```

Hiệu năng DDQN

- So sánh hiệu năng DDQN, DQN và người chơi trên Atari
- DDQN: điểm cao hơn DQN gốc
- Không phải lúc nào cũng đúng -> thử cả hai

PERF VS HUMAN



¹ <https://arxiv.org/abs/2303.11634>

Luyện tập nhé!

DEEP REINFORCEMENT LEARNING BẰNG PYTHON

Phát lại kinh nghiệm theo mức ưu tiên

DEEP REINFORCEMENT LEARNING BẰNG PYTHON



Timothée Carayol

Principal Machine Learning Engineer,
Komment

Không phải trải nghiệm nào cũng như nhau

- Experience Replay:
 - Lấy mẫu đồng đều có thể bỏ lỡ ký ức quan trọng
- Prioritized Experience Replay:
 - Gán mức ưu tiên cho mỗi trải nghiệm dựa trên sai số TD
 - Tập trung vào trải nghiệm có tiềm năng học cao



Prioritized Experience Replay (PER)

```
for step = 1 to T do:
    # Thực hiện hành động tối ưu theo hàm giá trị
    # Quan sát trạng thái kế tiếp và phần thưởng
    # Thêm chuyển tiếp vào bộ đệm replay
    # Gán mức ưu tiên cao nhất (1)
    # Lấy mẫu một lô chuyển tiếp trước đây
    # Dựa trên mức ưu tiên (2)
    # Tính sai số TD cho lô
    # Tính loss và cập nhật Q Network
    # Dùng trọng số lấy mẫu theo tầm quan trọng (4)
    # Cập nhật mức ưu tiên của các chuyển tiếp đã lấy mẫu
    # Tăng dần lấy mẫu theo tầm quan trọng theo thời gian. (5)
```

(1) Chuyển tiếp mới được thêm với mức ưu tiên cao nhất $p_i = \max_k(p_k)$

(2) Lấy mẫu chuyển tiếp i với xác suất

$$P(i) = p_i^\alpha / \sum_k p_k^\alpha \quad (0 < \alpha < 1)$$

(3) Chuyển tiếp đã lấy mẫu được cập nhật mức ưu tiên bằng sai số TD của chúng:

$$p_i = |\delta_i| + \epsilon$$

(4) Dùng trọng số lấy mẫu theo tầm quan trọng

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)} \right)^\beta \quad (0 < \beta < 1)$$

Triển khai PER

```
def __init__(self, capacity, alpha=0.6, beta=0.4, beta_increment=0.001, epsilon=0.001):  
    # Khởi tạo bộ đệm bộ nhớ  
    self.memory = deque(maxlen=capacity)  
    # Lưu tham số và khởi tạo mức ưu tiên  
    self.alpha, self.beta, self.beta_increment, self.epsilon = (alpha, beta, beta_increment, epsilon)  
    self.priorities = deque(maxlen=capacity)  
  
...
```

Triển khai PER

```
...  
  
def push(self, state, action, reward, next_state, done):  
    # Thêm trải nghiệm vào bộ đệm bộ nhớ  
    experience_tuple = (state, action, reward, next_state, done)  
    self.memory.append(experience_tuple)  
    # Đặt ưu tiên của chuyển tiếp mới thành mức tối đa  
    max_priority = max(self.priorities) if self.memory else 1.0  
    self.priorities.append(max_priority)  
  
...
```

Triển khai PER

```
def sample(self, batch_size):
    priorities = np.array(self.priorities)
    # Tính xác suất lấy mẫu
    probabilities = priorities**self.alpha / np.sum(priorities**self.alpha)
    # Chọn ngẫu nhiên các chỉ số mẫu
    indices = np.random.choice(len(self.memory), batch_size, p=probabilities)
    # Tính trọng số
    weights = (1 / (len(self.memory) * probabilities)) ** self.beta
    weights /= np.max(weights)
    states, actions, rewards, next_states, done = zip(*[self.memory[idx] for idx in indices])
    weights = [weights[idx] for idx in indices]
    states, actions, rewards, next_states, done = (zip(*[self.memory[idx] for idx in indices]))
    # Trả về tensor
    states = torch.tensor(states, dtype=torch.float32)
    ... # Lặp lại cho rewards, next_states, done, weights
    actions = torch.tensor(actions, dtype=torch.long).unsqueeze(1)
    return (states, actions, rewards, next_states, done, indices, weights)
```

Triển khai PER

```
...

def update_priorities(self, indices, td_errors: torch.Tensor):
    # Cập nhật mức ưu tiên cho các chuyển tiếp đã lấy mẫu
    for idx, td_error in zip(indices, td_errors.abs()):
        self.priorities[idx] = abs(td_error.item()) + self.epsilon

def increase_beta(self):
    # Tăng dần beta về 1
    self.beta = min(1.0, self.beta + self.beta_increment)
```

PER trong vòng lặp huấn luyện DQN

1. Mỗi bước:

1. Trong mã trước vòng lặp:

```
buffer = PrioritizedReplayBuffer(capacity)
```

2. Đầu mỗi tập:

```
buffer.increase_beta()
```

```
# Sau khi chọn hành động
buffer.push(state, action, reward,
            next_state, done)

...

# Trước khi tính sai số TD:
replay_buffer.sample(batch_size)

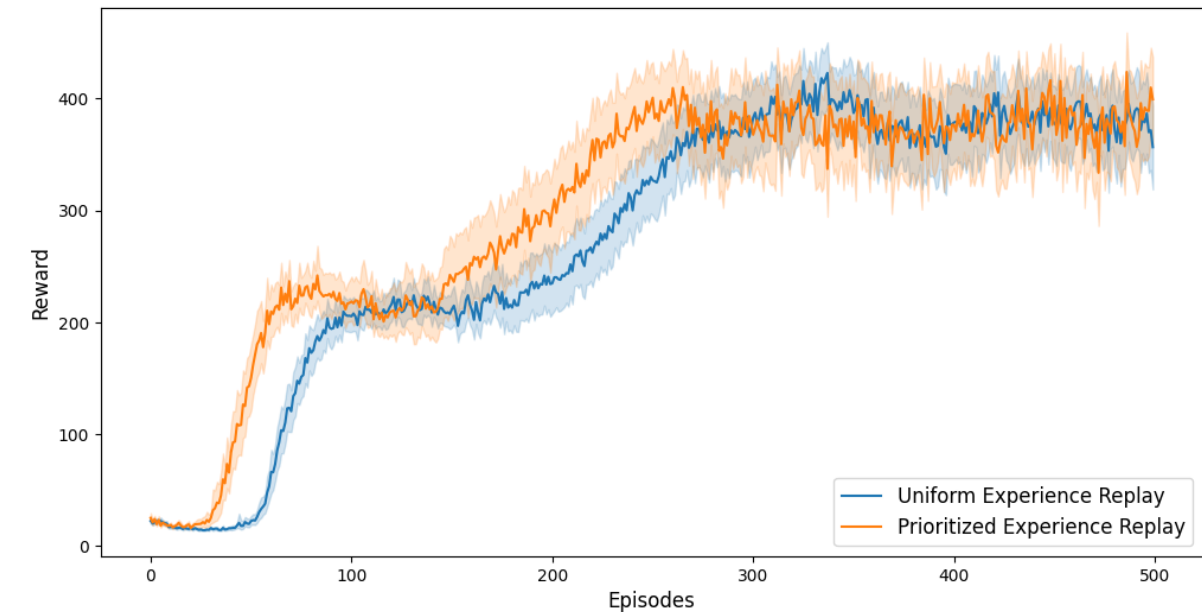
...

# Sau khi tính sai số TD
buffer.update_priorities(indices, td_errors)
loss = torch.sum(weights * (td_errors ** 2))
```

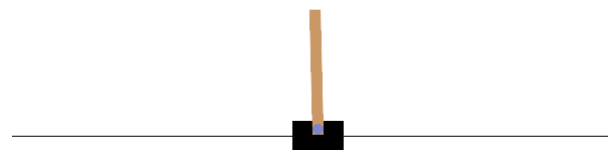
PER thực tế: Cartpole

100 lượt huấn luyện trong môi trường
Cartpole:

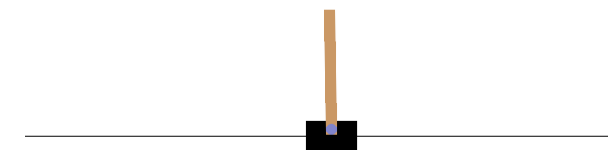
1. có Prioritized Experience Replay
 2. có Uniform Experience Replay
- Học nhanh hơn và hiệu năng tốt hơn với



Sau 100 epoch:



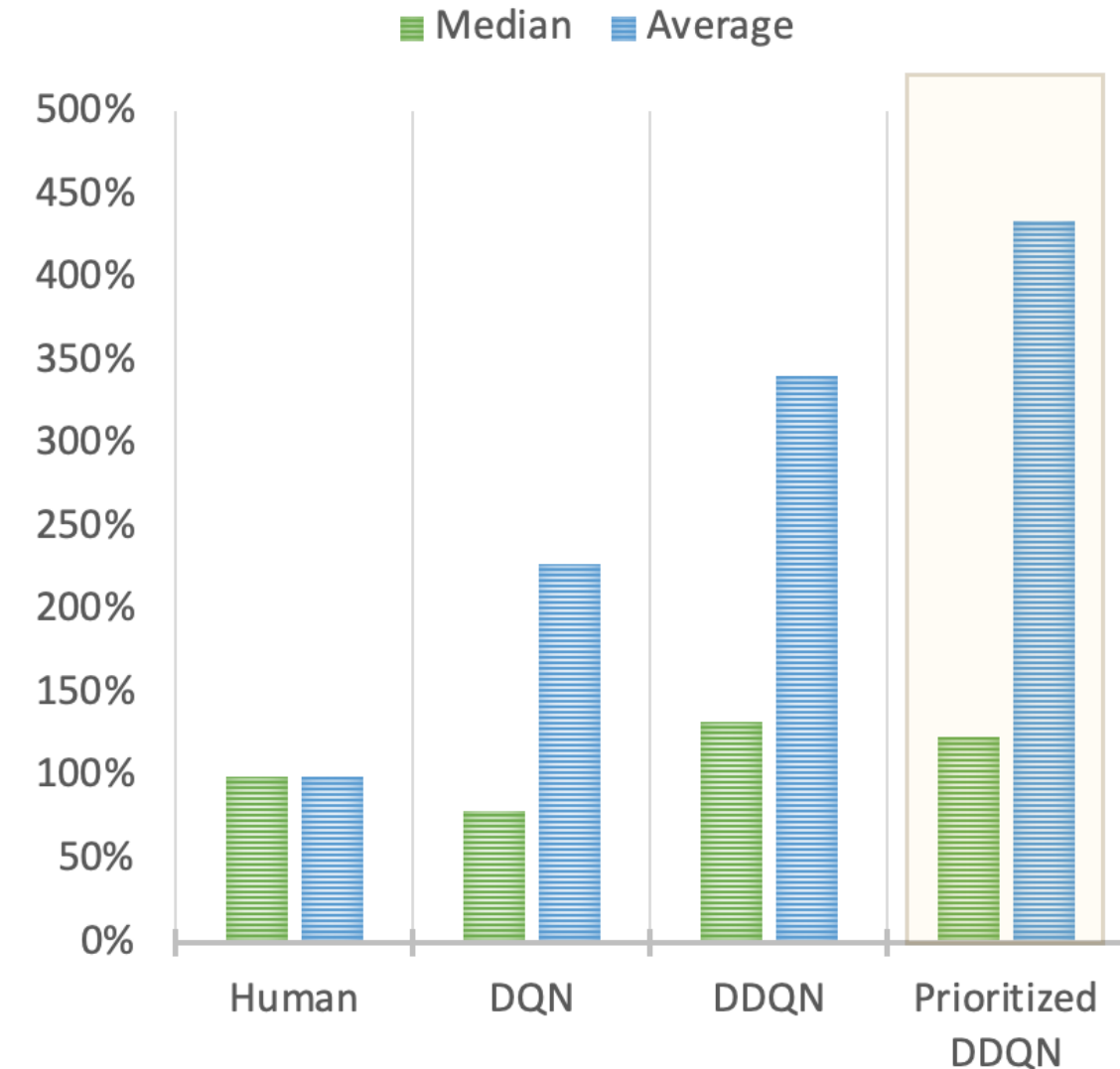
Sau 400 epoch:



PER thực tế: môi trường Atari

- PER tăng hiệu năng đáng kể trong môi trường Atari

PERF VS HUMAN



¹ <https://arxiv.org/abs/2303.11634>

Ayo berlatih!

DEEP REINFORCEMENT LEARNING BẰNG PYTHON